

Validity testing issues

By "validity testing" we mean SFINAE as well as some predicate type traits, like `std::is_convertible`, where the validity of certain language constructs can be tested without making the program ill-formed. This paper lists the known cases where the Standard leaves it unclear what a programmer can expect of a conforming implementation when testing the validity of certain constructs. While the problem used to be rather niche in C++, exploited only by seasoned C++ programmers, we expect that it will become more common with the addition of Concepts Lite. The constraint satisfaction mechanism in Concepts Lite exploits the same validity testing mechanism.

Most issues revolve around formalizing the notion of "immediate context", but our scope is a bit broader. We do not propose any solution at this point. Instead, our goal is to seek the Committee's guidance on how the listed issues are intended to be resolved.

Issues' description

The Standard gives us only partially formal description of what what is detectable in validity testing and what constitutes a "hard" error. We only get this text in section 14.8.2 and throughout section 20.13, in slight variations:

Access checking is performed as if in a context unrelated to subexpressions involved. Only the validity of the immediate context of the variable initialization is considered. [*Note*: The evaluation of the initialization can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]

But this leaves a number of cases unclear. For most of the issues listed here, we require an answer to the question, whether validity testing of a given language construct (expression or object initialization) should return false, return true, end in compilation-failure or the outcome be implementation-defined, or, in the worst case, unspecified.

Issue 1 — using a deleted function is in immediate context?

The question is: is using a deleted function in the immediate context of the tested expression? Or in other words, when I test expression validity, and this expression uses a deleted function, should validity testing return a negative answer, or should it be a hard error (that makes the entire program ill-formed)?

This question can be stated in yet another way: is the following program correct?

```
#include <type_traits>

struct Rational
{
    int num, den;
    constexpr Rational(int n, int d = 1) : num(n), den(d) {}
    constexpr Rational(double n) = delete;
};

template <typename T>
constexpr auto test_rational_convert(T v) -> decltype(Rational{v}, true) { return true; }

constexpr bool test_rational_convert(...) { return false; }

int main()
{
    static_assert(std::is_convertible<int, Rational>::value, "1");
    static_assert(!std::is_convertible<double, Rational>::value, "2");
    static_assert(test_rational_convert(5), "3");
    static_assert(!test_rational_convert(5.0), "4");
}
```

It tests in two ways (using a type trait and expression-based sfinae) whether certain conversion is disabled (by declaring function as deleted). The Standard is a bit ambiguous. 8.4.3 paragraph 2 says:

A program that refers to a deleted function implicitly or explicitly, other than to declare it, is ill-formed. [*Note*: This includes calling the function implicitly or explicitly and forming a pointer or pointer-to-member to the function. It applies even for references in expressions that are not potentially-evaluated. If a function is overloaded, it is referenced only if the function is selected by overload resolution. —*end note*].

Thus, taken literally, our sample program is ill-formed. On the other hand 14.8.2 paragraph 8 says:

An invalid type or expression is one that would be ill-formed, with a diagnostic required, if written using the substituted arguments. [*Note*: If no diagnostic is required, the program is still ill-formed. Access checking is done as part of the substitution process. —*end note*] Only invalid types and expressions in the immediate context of the function type and its template parameter types can result in a deduction failure.

As a side note, we have a general contradiction here: one part of the Standard says that some condition C1 makes a program ill-formed, but another part of the Standard says that C1 does not make a program ill-formed. Which is more important then?

Recommendation: We strongly feel that validity testing should detect the usage of deleted functions, and give a negative answer. This is really useful in unit-testing whether we have correctly disabled a dangerous usage of our class. GCC (6.1) and Clang (3.8) already do it. On MSVC (2015) assertion 2 fails: type trait `std::is_convertible<double, Rational>::value` returns true, as though it treated the marking of the function as deleted in non-immediate context.

Issue 2 — invalid constructs in variable templates

Is an error in initialization of a variable template detected in validation testing? Is it an "immediate context"? In other words, is the following supposed to be a well-formed program?

```
#include <string>

template <typename T>
const int r = T(1) / 2; // uses division operator

template <typename T>
auto half(T&&) -> decltype(r<T>)
{ return r<T>; }

int half(const std::string&)
{ return 0; }

int main()
{
    std::string s;
    half(s);
}
```

Both GCC and CLang treat the program as ill-formed. That is, initializer of a variable template we read is not an immediate context. MSVC (2015) does not support variable templates.

Issue 3 — expressions in function default arguments are in immediate context?

Next unanswered question: are errors in initializer expression of default function arguments in the immediate context of the tested expression? In other words, is the following program well-formed?

```
template <typename T, typename U = T>
void fun(T v, U u = U()) {}

void fun(...) {}

struct X
{
    X(int) {} // no default ctor
};

int main()
{
    fun (X(1));
}
```

Similarly, is the following program ill-formed?

```
template <typename U>
void fun(U u = U());

struct X
```

```
{
    X(int) {}
};

template <class T>
decltype(fun<T>()) g(int) { }

template void g(long) { }

int main() { g<X>({}); }
```

They look similar, but GCC happens to treat the former as ill-formed and the latter as well-formed. Clang and MSVC treat both programs as ill-formed.

Issue 4 — short circuiting allowed?

The following code snippet makes the program ill formed, and all compiler seem to accept that:

```
template <typename T>
struct tool_helper : T {}; // invalid if T is int

struct tool_x
{
    tool_x(long) {};

    template <typename T,
              typename U = typename tool_helper<T>::type> // T is int
    tool_x(T, U = {}) {};
};

bool b = std::is_convertible<int, tool_x>::value;
```

We get "hard" error when instantiating class template `tool_helper` which would derive from `int`. However, if we chane the test in the last line to:

```
bool b = std::is_convertible<long int, tool_x>::value;
```

Clang and GCC do no longer treat it as a hard error, and return `true`. I guess, this happens because when seeing the first declaration and only the first argument of the other declaration, they can immediately conclude, that the first one will be the best unambiguous match. So, they save time and just pick the first overload without inspecting the remaining arguments of the second overload, and they have no need to instantiate `tool_helper<int>`. According to the current wording, such short circuiting is not allowed: obviously the optimization changes an ill-formed program into a well formed one. If a programmer expects the compilation to fail (because he implements a tricky compile-time safety feature), he will be surprised. On the other hand, such optimization appears useful.

It seems, tah there are situation where an implementation may or may not need to instantiate a template in testing expression validity, depending on the quality of the implementation. The question is, should implementations be allowed to perform this kind of optimizations?

Issue 5 — checking function bodies allowed?

Currently, the Standard specifies that (unless you are using return type deduction), validity testing mechanism does not check function template bodies. This probably means that function bodies are not even instantiated. The question is, if a given implementation volunteers to instantiate some of the function template bodies and treat instantiation failures as "soft" errors, detectable by validity testing mecanism, should it be called standards-conforming or not?

Issue 6 — hard error outside immediate context are mandatory or optional?

Similarly, if in order to test the validity of an expression an implementation instantiates a class template, and we get an error during this instantiation, the implementation is *allowed* to stop the compilation at this point, and conclude that the program is ill-formed. But is it *required* to? Can an ultra-fast, ultra-diligent implementation implement a full template instantiation baacktracking if it chooses to? For instance, the following program (taken from issue [2496](#)) is legally ill-formed:

```
#include <type_traits>

template <typename T> struct B : T { };
template <typename T> struct A { A& operator=(const B<T>&); };

static_assert(std::is_assignable<A<int>, int>::value, "ERR");

int main() {}
```

This is because we have to instantiate `B<int>` to check if it is convertible from `int`. But because checking this is so simple (the definition of `B` is so small), an implementation could just inspect the tokens to conclude that specialization `B<int>` is illegal, and have the type trait return false. Could a standard-conforming implementation do that?

Issue 7 — any ill-formedness reason is diagnosable, or only some?

Is any possible potential error observed in the immediate context of a language construct under test turned into a "false" answer returned from validity testing mechanism, or are there kinds of errors that still render the entire program ill-formed even if observed in the immediate context? E.g., exceeding implementation limits, or perhaps something similar. Should validity testing only detect forming invalid types/expressions/object initializations; or check any error whatsoever?

Issue 8 — can implementations detect ill-formedness reason where diagnostic is not required?

Some invalid constructs can cause the the program to be ill-formed with no diagnostic required (e.g., ODR violation). The validity testing mechanism in overload resolution must not detect those as substitution failure. However, if a given implementation chooses to detect ODR violations, and issue diagnostics, is it also allowed to use this information in validity testing mechanism?

Issue 9 — should SFINAE and type traits always give the same result?

Practically any predicate type-trait, like `is_assignable` can be equivalently tested using a `decltype` in function signature:

```
template <typename T>
auto is_copy_assignable_impl(int)
    -> decltype(((void)(std::declval<T>() = std::declval<const T>()), std::true_type{}));

template <typename T>
auto is_copy_assignable_impl(long)
    -> std::false_type;

template <typename T>
struct is_copy_assignable2 : decltype(is_copy_assignable_impl<T>(1)) {};
```

Meta function `is_copy_assignable2` defined as above is equivalent to `std::is_copy_assignable`. But if implementations are given certain freedom in deciding when to give a false positive, or when to prematurely terminate the compilation (rather than returning a true-false answer, it may result in situations where choosing between `is_copy_assignable2` and `std::is_copy_assignable` might give different results. The question is, should the Standard require (in the face of other kinds of implementation QOI freedom) that behaviour of SFINAE and corresponding type traits be consistent?

Currently, the Standard requires that only errors with diagnostic required can cause the validity testing in SFINAE return false. That is, if substitution fails due to ODR violation (diagnostic not required) this causes the validity testing mechanism to return true. In contrast, validity testing in type traits is required to return false whenever a tested expression is ill-formed, regardless of if diagnostic is required or not. We suppose this is an oversight.

Recommendation: We strongly feel that both validity testing context should always be interchangeable and give the same effects.

Summary

The following summarizes the questions that we would like CWG to give us directions for. This will help come up with the proposed wording changes in the future.

Q1. Is using a deleted function in the immediate context and a detectable condition? (Recommendation: yes.)

Q2. Are invalid constructs in variable template initialization in the immediate context, and a detectable condition?

Q3. Are expressions in function default arguments' initializers in the immediate context and a detectable condition?

Q4. Are implementations allowed to skip substitution of some overloads, if they can prove otherwise that these overloads would have never been chosen, or cause ambiguous result?

Q5. Can implementations check function bodies in validity testing, if they want to?

Q6. Can implementations check other errors in non-immediate context during validity testing, if they want to?

Q7. Are there kinds of ill-formedness reasons that we want to remain undetectable by validity testing?

Q8. Can implementations detect ill-formedness reason where diagnostic is not required in validity testing, if they want to?

Q9. Should all forms of validity testing (overload resolution, type traits, concept satisfaction) always have consistent results?

(Recommendation: yes).

The recommendation, that we feel strong about is only about the first and the last question. (1). Deleted functions should be detectable in validity testing. This is easily implementable, and allows writing compile-time tests whether certain dangerous conversions have been prevented. (9). SFINAE and type traits need to be consistent, so that changing the technique for disabling overloads should not affect the program behavior.

Acknowledgements

A number of people helped in creating the shape of this paper and in the analysis of the problems. In particular, Daniel Krügler and Tomasz Kamiński